

In most cases, a change to an applications features requires a change to the data it stores $\rightarrow$ store a new field or record type or existing data needs to be presented in a new way



evolvability

Schema changes often mean application code changes, but these cannot happen instantaneously:

- rolling updates to ensure new version is running smoothly
- for client side updates, user may not install the newest update

$\therefore$ new schema + old schema and new code + old code may coexist in the system at the same time $\rightarrow$ need compatibility

Backward Compatibility - old schema works with new code

Forward Compatibility - new schema works with old code

Formats for encoding data:

Programs work with data in two different representations:

1. In memory, data is kept in datastructures that make operations efficient
2. When writing data to file or sending it over network, must encode the data in a self-contained sequence of bytes



translation of data from memory to a byte sequence is called **encoding / serialization** and the reverse is **decoding / deserialization**

JSON, XML, CSV are good enough for many purposes

Thrift and Protocol Buffers:

- binary encoding libraries
- require schemas for any data that is encoded
- come with code generation tool that uses the schema to produce classes that implement schema in various languages, can also use this code to encode/decode records of schema

Schema Evolution :

- An encoded record is just the concatenation of encoded fields, where each field is identified using its tag number
- If a field value is not set, it is omitted from the encoded record

∴

old code reading data written by new code: **forward**

- just skips fields with tag numbers it doesn't recognize

new code reading data written by old code: **backward**

- as long as each field has a unique tag number, new code can always read old data

Avro :

- another binary encoding format
- we use reader and writer schemas, these schemas don't have to be the same, just compatible

Schema evolution :

- you can only add / remove a field from the schema that has a default value

Dataflow through Databases:

- process that writes to DB encodes the data, process that reads from DB decodes it
- schema evolution allows the entire database to appear as if it was encoded with a single schema even though the underlying storage may contain records encoded with various historical versions of the schema

Dataflow through Services:

Remote Procedure Calls (RPC) - tries to make a request to a remote network service look the same as calling a function or method in your programming language

- Thrift and Avro come with RPC support included, gRPC is an RPC implementation built on Protocol Buffers

Message - Passing Dataflow :

asynchronous message - passing systems :

- clients request (message) is delivered to another process with low latency
- message goes via an intermediary called **message broker**, which stores the message temporarily

Advantages :

- can act as buffer if recipient unavailable
- automatically redeliver messages to a process that has crashed
- avoid needing to know IP address and port #
- one message to several recipients
- decouples the sender from receiver

Sender sends messages and forgets about them

Message Brokers

- Kafka, RabbitMQ, netQ, NATS, HornetQ

used as follows : one process sends a message, to a named **queue** or **topic** and the broker ensures that the message is delivered to one or more **consumers** or **subscribers** of the topic

Distributed Actor Framework:

actor model - programming model for concurrency in a single process, encapsulates logic into **actors** rather than dealing with threads.

actors - represents one client or entity, may have some local state (not shared with any other actors). and sends|receives asynchronous messages to/from other actors to communicate; only processes one message at a time

distributed actor frameworks

- used to scale an application across multiple nodes
- if communicating with actors on different nodes, message is transparently encoded into a byte-sequence and sent over the network
- combines the job of a message broker with the actor programming model into one unified system